Deeparture V2

Future-Proof Architecture Principles - Version 1.0

Project-wide developer rules for all phases. These are primarily architecture constraints, not a request to build every future feature now.

Version 1.0

This document sits above the individual Deeparture V2 phase specifications.

Its purpose is to prevent short-term implementation choices from making future catalogue imports, availability feeds, operator integrations, agent tools, mobile apps, pricing models, or external APIs expensive to add later.

*These are primarily **architecture constraints**, not a request to build every future feature now.*

When a phase does not yet require a feature, the developer should preserve a clean extension point rather than implement the full feature early.

Deeparture should be built as a **source-independent travel inventory platform**, not as a collection of pages or spreadsheet-specific workflows.

The core system should not care whether data originally came from:

```
Admin form
Deeparture XLSX
CSV
Operator spreadsheet
Operator Portal
Google Sheet
JSON API
XML feed
Webhook
SFTP/file feed
Website extraction
Legacy migration
Specialist liveaboard data provider
```

All sources should ultimately pass through normalized Deeparture services and data contracts.

These principles should not be used to unnecessarily enlarge the current development phase.

Use three categories:

```
A. MUST DESIGN FOR NOW
Data structures, service boundaries, IDs, contracts, security and extension points.

B. IMPLEMENT WHEN THE PHASE REQUIRES IT
UI, workflows, connectors, automation and operational tools.

C. FUTURE INTEGRATION
External provider/API-specific implementations.
```

A developer should not build a future integration merely because this document mentions it.

The goal is to make future integrations **additive**, not require a redesign.

Every important entity must have a permanent Deeparture internal identifier.

Examples:

```
Organisation
User
Membership
Vessel
Resort
Itinerary
Departure
Cabin Type
Room Type
Package
Destination
```

```
Port
Price
Availability Record
Enquiry
Booking
Import Batch
External Source
```

Do not use names, slugs, URLs, spreadsheet row numbers, or operator-specific IDs as the primary identity of a Deeparture record.

Names and slugs can change.

Internal IDs should not.

Deeparture will eventually receive the same entity from multiple systems.

Maintain a provider/external-reference mapping layer.

Conceptual example:

```
Deeparture Vessel ID: 82fa...
Provider: booking_platform_x
External Object Type: vessel
External ID: 4172
```

A provider mapping should be able to link external identifiers to:

```
organisation
vessel
resort
itinerary
departure
cabin type
room type
package
availability inventory
```

This mapping must remain separate from the human-readable vessel/operator name.

This is important for:

- API integrations
- live availability feeds
- operator booking systems
- spreadsheet imports
- website extraction
- legacy migration
- deduplication

Public/internal application APIs should remain explicitly versioned.

Example:

```
/api/v2/...
```

Do not make future breaking changes silently inside an existing API contract.

Future customer apps, operator portals, agent tools and integrations may depend on a stable API.

Where possible:

```
additive change -> remain in current API version
breaking change -> new version or compatibility layer
```

Every standard Deeparture import template/contract should have a schema/template version.

Example:

```
deepartureImportSchemaVersion: 2
```

This applies to:

- XLSX templates
- CSV formats
- normalized import DTOs
- future API bulk imports

Old templates should either remain readable or fail with a clear compatibility message.

Do not make spreadsheet-header changes silently break historical files.

CSV and XLSX are source adapters, not the architecture.

Required pattern:

```
CSV / XLSX -------------------\
Operator spreadsheet ----------\
API / JSON ---------------------> Normalized Catalogue Import Schema
Website extraction ------------/ |
Legacy migration -------------/ v
Validation
|
v
Match / Duplicate Check
|
v
Preview
|
v
Admin Confirmation
|
v
Catalogue Service
|
v
V2 Database
```

A source parser must not contain catalogue business logic that future adapters would need to duplicate.

Availability must also remain source-independent.

Required pattern:

```
Admin manual update -----------\
CSV / XLSX ---------------------\
Operator Portal ----------------> Normalized Availability Update
API / JSON --------------------/
Webhook ----------------------/
Google Sheet ----------------/
Provider feed --------------/
|
v
Availability Service
|
v
Validation / Matching / Upsert
|
v
Effective Availability
|
v
Search / Public API / Admin
```

Search/public consumers must not care which source supplied the availability.

Availability data should retain:

```
source
sourceUpdatedAt
receivedAt
stable inventory identity
```

```
effective/resolved value
```

Do not call data "live" unless the source/freshness rules justify that description.

Repeated submissions must be safe.

Examples:

- importing the same row twice
- receiving the same webhook twice
- retrying an API request after a timeout
- reprocessing an availability file
- retrying a booking/enquiry event

The same logical update should not create duplicate entities or duplicate transactions.

Where appropriate, support:

```
idempotency key
external event ID
stable source reference
upsert key
```

Important commercial records should normally be archived or deactivated rather than physically deleted.

Examples:

- organisation
- vessel
- resort
- itinerary
- departure
- pricing
- enquiries
- bookings
- provider mappings
- imports

Historical relationships matter for support, reporting, disputes, auditing and future data reconciliation.

Permanent deletion may still exist for appropriate data/privacy workflows, but routine admin "delete" should not casually destroy history.

Do not reduce travel pricing to a single numeric `price` field.

The architecture should be capable of representing:

```
amount
currency
pricing basis
product/cabin/room type
base price
departure/date override
single supplement
mandatory fees
taxes
discount/promotion
future net rate
future agent rate
future commission
```

Possible pricing bases include:

```
per person
per cabin
per room
per package
per night
per charter
```

Not all layers need to be implemented immediately.

The data/service design must not prevent them later.

Retain the supplier/native currency and amount.

Do not permanently convert all prices into USD.

Where cross-currency search/comparison is required, maintain:

```
native amount
native currency
normalized/search amount
conversion rate/reference
conversion timestamp
```

The displayed commercial price should remain traceable to the supplier's original currency.

Do not assume the server timezone is the business timezone.

Store timestamps in a consistent machine-safe format and retain the relevant local timezone where business meaning depends on local time.

Important examples:

- departure start/end
- booking cutoff
- cancellation deadline
- source synchronization time
- offer expiry
- claim invitation expiry
- manual override expiry

A departure date should not change because a user or server is in a different timezone.

Liveaboard availability must be able to exist at more than one level.

Examples:

```
departure status
overall spaces remaining
cabin-type availability
specific cabin inventory later if required
```

A departure may be available while one cabin type is sold out.

The model should not force a single Boolean:

```
available = true/false
```

when the source can provide richer data.

The system must be capable of distinguishing:

```
fresh
aging
stale
unknown
```

Stale must not automatically mean sold out.

When several sources report the same inventory, Deeparture must be able to support:

- configurable source priority
- source timestamps
- conflict detection
- manual override
- override expiry
- update history

Do not hard-code one provider as universally authoritative.

Important business actions should emit internal application events.

Examples:

```
catalogue.listing.created
catalogue.listing.published
catalogue.listing.restricted

organisation.claim.invited
organisation.claim.completed

departure.created
departure.availability.updated
departure.price.updated

enquiry.created
enquiry.approved
enquiry.operator_accepted

booking.created
booking.status_changed
```

This does not require a complex event-streaming platform now.

Initially, a clean internal event service is sufficient.

Later the same events can trigger:

- email
- in-app notification
- webhooks
- search indexing
- analytics
- CRM
- agent notifications

Business logic should not be duplicated across those consumers.

Slow or retryable operations should not permanently depend on a browser request remaining open.

Future examples:

- 5,000-row catalogue import
- website extraction
- currency refresh
- availability synchronization
- operator API polling
- email batches

- media processing
- search indexing

The architecture should allow these to become background jobs/workers.

Phase-specific code does not need to introduce a heavy queue system before one is needed.

Provider-specific logic belongs in adapters/connectors.

Examples:

```
SharcConnector
BookingPlatformConnector
OperatorApiConnector
GoogleSheetConnector
XmlFeedConnector
SftpConnector
WebsiteExtractor
```

Core catalogue, pricing and availability services should not contain provider-specific field names or API assumptions.

Removing one provider should not require rewriting the core Deeparture models.

Do not model permissions as only:

```
admin
operator
```

Use organisations + memberships + roles/capabilities.

Future roles may include:

```
organisation owner
admin
content editor
reservations
accounts
viewer
Deeparture internal admin
agent owner
agent staff
```

A user should be capable of belonging to more than one organisation.

Server-side permission checks remain mandatory.

A vessel or resort must have its own identity independent of its current operator/management organisation.

Vessels and resorts can:

- change owner
- change operator
- change management company
- change brand
- change name

Avoid defining the product identity as:

```
operator + vessel name = vessel
```

Long-term, the architecture should be capable of representing a dated relationship:

```
Vessel X
Operator A: validFrom ... validTo ...
Operator B: validFrom ... validTo ...
```

The complete historical UI is not required now.

The data design should not make this impossible later.

Public URLs are presentation/SEO identifiers, not database identity.

If a vessel/resort/operator slug changes:

- the entity ID stays the same
- old public URLs should be capable of redirecting to the new URL

This protects SEO and external links when products are renamed.

Do not make catalogue records depend permanently on one image-storage provider or local filesystem path.

Store media as managed references/records.

The system should later be able to move between:

- local/staging storage
- object storage
- CDN
- media-management service

without rewriting every vessel/resort document.

Media metadata should be able to support:

```
owner/source
permission/licence note
alt text
caption
sort order
hero image
active/archived
```

MongoDB remains the system of record.

Search should be a replaceable read/index layer.

Deeparture may begin with MongoDB queries/Atlas capabilities and later use a specialized search engine.

Do not put irreplaceable business data only inside a search index.

Filterable concepts should use IDs/controlled definitions rather than uncontrolled free text.

Examples:

- destination
- country
- region
- embarkation port
- dietary feature
- accessibility feature
- facility
- dive feature
- cabin type
- trip type

Aliases/synonyms can exist, but they should resolve to a canonical record.

Facilities, accessibility, dietary and other evolving features should use an extensible attribute-definition system.

Adding:

```
Starlink
accessible tender
camera workstation
vegan chef
rebreather support
```

should not normally require adding new hard-coded fields to every Vessel/Resort schema.

Attributes should define:

```
name
slug
category
data type
appliesTo
filterable
publicVisible
active
sortOrder
```

Enquiry, booking, claim, import and approval workflows should have explicit states and permitted transitions.

Do not rely on a loose collection of Booleans such as:

```
approved=true
accepted=true
confirmed=false
```

when those values can become contradictory.

Example:

```
enquiry:
new -> admin_approved -> operator_sent -> operator_accepted -> confirmed
```

The exact states may evolve, but transitions should be explicit and auditable.

Deeparture should be able to answer:

```
Where did this data come from?
When was it last checked?
Was it verified by the operator?
Was it verified by Deeparture?
Was it imported, manually entered, or supplied by an API?
```

At minimum preserve record-level provenance.

Field-level provenance can be added where business value justifies it.

Important administrative/commercial actions should be traceable.

Examples:

- listing published
- listing restricted
- operator verified
- price changed
- availability overridden
- import committed
- claim invitation sent
- enquiry status changed

Never place passwords, tokens, API secrets or unnecessary personal data into audit payloads.

Production data-model changes should use repeatable migration scripts/processes.

Avoid:

```
"Open MongoDB and manually edit these 400 records."
```

Every migration should be:

- version-controlled
- repeatable or safely guarded
- testable on staging
- logged
- backed up before production execution

New capabilities should be capable of limited rollout.

Examples:

- agent pricing
- external availability connector
- new booking workflow
- operator self-publishing
- API access
- PWA feature

A feature may be enabled for:

```
staging only
specific organisation
specific role
percentage/cohort
all users
```

Do not require a full redeploy/rewrite merely to hide an unfinished feature.

Deeparture must become diagnosable as integrations increase.

At minimum maintain:

- health endpoint
- structured application logs
- import error logs
- integration/sync status
- job status when background jobs exist
- clear timestamps
- request/error correlation where practical

Eventually the system should answer:

```
Why is Operator X's availability stale?
When did Provider Y last sync successfully?
Which import created this vessel?
Why did this webhook fail?
```

Backups are only useful if restoration works.

Before production V2 becomes business-critical:

- automatic database backups
- media backup/replication where appropriate

- documented restore process
- periodic restore test
- clear staging vs production separation

Do not test restore procedures for the first time during an incident.

Secrets must not be committed to Git.

Examples:

- MongoDB credentials
- JWT secrets
- SMTP passwords
- API keys
- webhook signing secrets

Use deployment secret/environment storage.

External connector credentials should remain separate from normal catalogue records and must not be written into audit logs.

Use least-privilege access where practical.

When future providers push data to Deeparture:

- verify signatures/shared secrets where provider supports them
- protect against replay where practical
- use idempotency/event IDs
- validate payloads
- log accepted/rejected event IDs
- return appropriate retry-safe responses

Never trust an incoming webhook solely because it knows a public URL.

Future APIs/connectors will fail temporarily.

Plan for:

- timeout
- retry with limits/backoff
- provider rate limits
- partial provider outage
- stale data
- malformed records
- duplicate events

An external provider outage must not take down the public Deeparture website.

Do not bake English presentation text into database structures where codes/structured values are more appropriate.

The architecture should allow future:

- translated public descriptions
- localized dates
- localized number formats
- currencies

- multilingual attribute labels

Full translation is not required in the current phases.

Commercial catalogue data and customer/booking personal data should remain logically distinct.

Collect only personal data required for the workflow.

A catalogue import should not accidentally expose customer data.

Organisation users should only access records permitted by their memberships/roles.

Future agent access must not automatically expose another agent's client records.

Keep development/staging/production clearly separated.

At minimum:

```
separate environment variables
separate database
separate email behaviour
separate external integration credentials
clear environment indicator
```

Staging must not accidentally send production emails, mutate production inventory, or use production webhook credentials.

Core Deeparture services should be usable by more than the Admin UI.

Future consumers include:

```
Customer web frontend
PWA
Native app
Operator Portal
Agent Portal
Partner API
Internal Admin
Background workers
```

Business logic belongs in backend/domain services rather than being duplicated in frontend components.

These principles are cross-phase, but the implementation emphasis is:

Area	Primary phase
Core IDs / organisations / base models	Phase 1
Normalized catalogue import	Phase 2A
Normalized availability foundation	Phase 2A
Import/admin management	Phase 2A / 2B
Availability freshness/conflicts/history	Phase 2B
Operator editing / direct availability updates	Phase 3
Customer search/display	Phase 4
External feeds/APIs/webhooks/jobs	Phase 5
Agent/net pricing/agency permissions	Phase 6

This table does not prevent an earlier phase from creating an extension point needed by a later phase.

When choosing between a quick implementation and a more extensible one, ask:

Can the next data source, provider, user role or frontend be added as an adapter/consumer of the existing core services, or would it require replacing the current implementation?

Prefer the design where future capability can be added as an adapter, mapping, service extension, event consumer or new UI.

Avoid speculative complexity that has no plausible use.

Before signing off any future phase, confirm:

- permanent internal IDs remain intact
- no core business logic has moved into spreadsheet/provider-specific code
- normalized catalogue/availability service boundaries remain intact
- external references remain separate from Deeparture IDs
- permissions remain server-side
- no production secrets are committed
- API contracts remain versioned
- migrations are controlled
- old data can still be interpreted
- new functionality did not create a duplicate subsystem
- future external connectors can still be added without rewriting core catalogue/search
- production/staging remain isolated

This document is **not** permission to build every possible future feature now.

It exists so each phase can remain appropriately sized while still contributing to one coherent long-term Deeparture platform.

The preferred pattern is:

```
Design extension point now
Implement required feature in its correct phase
Add future integration as an adapter
Avoid rebuilding the core
```